Trees & Graphs - Programs & Solutions

1. Write a C program that uses functions to implement depth-first-graph traversal algorithms using Arrays.
2. Write a C program that uses functions to implement depth-first-graph traversal algorithms using Pointers/ singly linked list.
3. Write a C program that uses functions to implement breadth-first-graph traversal algorithms using Arrays.
4. Write a C program that uses functions to implement breadth-first-graph traversal algorithms using Pointers/ singly linked list.
5. Write a C program that uses functions to perform the following operations on binary tree using arrays:
    a. Create a binary tree of integers
    b. Traverse above binary tree recursively
        i. in-order
        ii. pre-order
        iii. post-order

6. Write a C program that uses functions to perform the following operations on binary tree using arrays:
    a. Create a binary tree of characters
    b. Traverse above binary tree recursively
        i. in-order
        ii. pre-order
        iii. post-order

7. Write a C program that uses functions to perform the following operations on binary tree using double linked list:
    a. Create a binary tree of integers
    b. Traverse above binary tree non-recursively
        i. in-order
        ii. pre-order
        iii. post-order

8. Write a C program that uses functions to perform the following operations on binary tree using double linked list:
    a. Create a binary tree of integers
    b. Traverse above binary tree recursively
        i. in-order
        ii. pre-order
        iii. post-order

9. Write a C program that uses functions to perform the following operations on binary search tree using double linked list:

a. Create a binary search tree of integers
b. Traverse above binary search tree recursively
    i. in-order
    ii. pre-order
    iii. post-order

Trees & Graphs - Solutions

1. **Write a C program that uses functions to implement depth-first-graph traversal algorithms using Arrays.**

```
//wap for DFS-Adjacency Matrix using 2-d arrays at run time
#include<stdio.h>
#include<conio.h
>
#include<stdlib.h
>
#include<ctype.h>

#define max1 20
#define true 1
#define false 0


        int g[max1][max1],v[max1];
        int v1,v2;
        int n;
        void gdfs(void);
        void create(void);
        void dfs(int v3);
        void traverse(void);
        void error(char *msg);


void main(void)
{
int v1;
clrscr();
gdfs();
create();
printf("\n\n\t\t The DFS-Adjacency Matrix for the graph is \n");
traverse();
printf("\n Enter the vertex from which u want to traverse: ");
```

```
scanf("%d",&v1);
if(v1>=max1)
```

```
error("invalid vertex");
printf("\n\n\t The Depth First Search (DFS) of the graph is \n");
dfs(v1);
getch();
}//main

void gdfs(void)
{
for(v1=0;v1<max1;v1++)
v[v1]=false;
for(v1=0;v1<max1;v1++)
for(v2=0;v2<max1;v2++)
g[v1][v2]=false;
}//gdfs


void traverse(void)
{
for(v1=0;v1<n;v1++)
{
for(v2=0;v2<n;v2++)
printf(" %d ",g[v1][v2]);
printf("\n");
}
}//traverse

void error(char *msg)
{
printf("\n %s ",msg);
printf("press any key to abort\n");
getch();
exit(1);
}//error

void create(void)
{
int ch, v1, v2, undirected;
undirected=true;
n=0;
printf("\n\t\t This is a program to create a graph");
printf("\n\t\t and the display is in depth first manner");
printf("\n Press d/D for directed and u/U for undirected graph \n");
ch=toupper(getche());
if(ch=='D')
undirected=false;
do
```

```
{
printf("\n Enter the edge of a graph by two vertices \n");
printf("\n (And for termination type -99) \n");
scanf("%d%d",&v1,&v2);
if(v1==-99)
break;
if(v1>=max1 || v2>=max1)
error("invalid vertex value \n");
else
g[v1][v2]=true;
if(undirected)
g[v2][v1]=true;
n++;
}while(1);
}//create

void dfs(int v3)
{
int v4;
printf("\n %d ",v3);
v[v3]=true;
for(v4=0;v4<n;v4++)
if(g[v3][v4]==true && v[v4]==false)
dfs(v4);
}//dfs
```

2. **Write a C program that uses functions to implement depth-first-graph traversal algorithms using Pointers/ singly linked list.**

```
//wap for DFS - Adjacency List using singly linked list and templates at run time
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

#define max1 20
#define true 1
#define false 0

        struct node
        {
        int vertex;
        struct node *next;
        };
        struct node *head[max1];
        int visit[max1];
```

```
        void init(void);
        void create(void);
        void dfs(int v1);


void main(void)
{
char ans;
int v1;
clrscr();
init();
create();
printf("\n\n\t\t The DFS-Adjacency List for the graph is \n");
printf("\n Enter the vertex from which u want to traverse: ");
scanf("%d",&v1);
printf("\n\n\t The Depth First Search (DFS) of the graph is \n");
dfs(v1);
getch();
}//main

void init(void)
{
int v1;
for(v1=0;v1<max1;v1++)
visit[v1]=false;
for(v1=0;v1<max1;v1++)
head[v1]=NULL;
}//init


void create(void)
{
int v1, v2;
char ans='y';
struct node *temp, *first;
printf("\n\n Enter the vertices number beginning with 0");
do
{
printf("\n Enter the vertices/edge of a graph: \n");
scanf("%d%d",&v1,&v2);
temp=(struct node *)malloc(sizeof(struct node));
if(temp==NULL)
printf("\n Insufficient memory\n");
temp->vertex=v2;
temp->next=NULL;
first=head[v1];
```

```
if(first==NULL
)
head[v1]=temp;
else
{
while(first->next !=
NULL) first=first->next;
first->next=temp;
}
temp=(struct node *)malloc(sizeof(struct node));
if(temp==NULL)
printf("\n Insufficient memory\n");
temp->vertex=v1;
temp->next=NULL
; first=head[v2];
if(first==NULL)
head[v2]=temp;
else
{
while(first->next !=
NULL) first=first->next;
first->next=temp;
}

printf("\n Want to add more vertices/edges: (y/n) ");
ans=getche();
}while(ans=='y');
}//create


void dfs(int v1)
{
struct node *first;
printf("\n %d ",v1);
visit[v1]=true;
first=head[v1];
while(first!=NULL)
if(visit[first->vertex]==false)
dfs(first->vertex);
else
first=first->next;
}//dfs
```

3. **Write a C program that uses functions to implement breadth-first-graph traversal algorithms using Arrays.**
   //wap for BFS - Adjacency Matrix using 2-d arrays at run time

```
#include<stdio.h>
#include<conio.h
>
#include<stdlib.h
>
#include<ctype.h>

#define max1 20
#define true 1
#define false 0


        int g[max1][max1],visit[max1], q[max1];
        int v1,v2;
        int front, rear;
        int n;

        void create(void);
        void bfs(int v);
        void traverse(void);
        void error(char *msg);
        void gbfs(void);

void main(void)
{
int v1;
clrscr();
gbfs();
create();
printf("\n\n\t\t The BFS-Adjacency Matrix for the graph is \n");
traverse();
printf("\n Enter the vertex from which u want to traverse: ");
scanf("%d",&v1);
if(v1>=max1)
error("invalid vertex");
printf("\n\n\t The Breadth First Search (BFS) of the graph is \n");
bfs(v1);
getch();
}//main

void gbfs(void)
{
for(v1=0;v1<max1;v1++)
for(v2=0;v2<max1;v2++)
g[v1][v2]=false;
for(v1=0;v1<max1;v1++)
visit[v1]=false;
```

}//gbfs

```
void traverse(void)
{
for(v1=0;v1<n;v1++)
{
for(v2=0;v2<n;v2++)
printf(" %d ",g[v1][v2]);
printf("\n");
}
}//traverse

void error(char *msg)
{
printf("\n %s \n",msg);
printf("press any key to abort\n");
getch();
exit(1);
}//error

void create(void)
{
int ch, flag;
flag=true;
n=0;
printf("\n\t\t This is a program to create a graph");
printf("\n\t\t and the display is in breadth first manner");
printf("\n Press d/D for directed and u/U for undirected graph \n");
ch=toupper(getche());
if(ch=='D'
)
flag=false;
do
{
printf("\n Enter the edge of a graph by two vertices \n");
printf("\n (And for termination type -99) \n");
scanf("%d%d",&v1,&v2);
if(v1==-99)
break;
if(v1>=max1 || v2>=max1)
error("invalid vertex value \n");
else
g[v1][v2]=true;
if(flag)
g[v2][v1]=true;
n++;
}while(1);
}//create
```

```
void bfs(int v)
{
int v3;
visit[v]=true;
front=rear=-1;
q[++rear]=v;
while(front!=rear)
{
v=q[++front];
printf("\n %d ",v);
for(v3=0;v3<n;v3++)
{
if(g[v][v3]==true && visit[v3]==false)
{
q[++rear]=v3;
visit[v3]=true;
}
}
}
}//bfs
```

4. **Write a C program that uses functions to implement breadth-first-graph traversal algorithms using Pointers/ singly linked list.**

```
//wap for BFS - Adjacency List using singly linked list and templates at run time
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

#define max1 20
#define true 1
#define false 0

        struct node
        {
        int vertex;
        struct node *next;
        };
        struct node *head[max1];
        int visit[max1];
        int q[max1];
        int front, rear;

        void init(void);
```

```
        void create(void);
        void bfs(int v1);

void main(void)
{
char ans;
int v1;
clrscr();
init();
create();
printf("\n\n\t\t The BFS-Adjacency List for the graph is \n");
printf("\nEnter the vertex from which u want to traverse: ");
scanf("%d",&v1);
printf("\n\t The Breadth First Search (BFS) of the graph is \n");
bfs(v1);
getch();
}//main

void init(void)
{
int v1;
for(v1=0;v1<max1;v1++
) visit[v1]=false;
front=rear=-1;
for(v1=0;v1<max1;v1++
) head[v1]=NULL;
}//init


void create(void)
{
int v1, v2;
char ans='y';
struct node *temp, *first;
printf("\n\n Enter the vertices number beginning with 0");
do
{
printf("\n Enter the vertices/edge of a graph: \n");
scanf("%d%d",&v1,&v2);
temp=(struct node *)malloc(sizeof(struct node));
if(temp==NULL)
printf("\n Insufficient memory\n");
temp->vertex=v2;
temp->next=NULL;
first=head[v1];
if(first==NULL)
```

```
head[v1]=temp;
else
{
while(first->next !=
NULL) first=first->next;
first->next=temp;
}

temp=(struct node *)malloc(sizeof(struct node));
if(temp==NULL)
printf("\n Insufficient memory\n");
temp->vertex=v1;
temp->next=NULL
; first=head[v2];
if(first==NULL)
head[v2]=temp;
else
{
while(first->next !=
NULL) first=first->next;
first->next=temp;
}

printf("\n Want to add more vertices/edges: (y/n) ");
ans=getche();
}while(ans=='y');
}//create


void bfs(int v1)
{
int i;
struct node *first;
q[++rear]=v1;
while(front!=rear)
{
i=q[++front];
if(visit[i]==false)
{
printf("\n %d ",i);
visit[i]=true;
}
first=head[i];
while(first!=NULL)
{
if(visit[first->vertex]==false)
```

```
q[++rear]=first->vertex;
first=first->next;
}
}
}//bfs
```

5. **Write a C program that uses functions to perform the following operations on binary tree using arrays:**
    a. **Create a binary tree of integers**
    b. **Traverse above binary tree recursively**
        i. **in-order**
        ii. **pre-order**
        iii. **post-order**

```
//wap for binary tree - integers using
arrays #include<stdio.h>
#include<stdlib.h
>
#include<conio.h
> #define max1
100

struct node
{
    struct node *lchild;
    int data;
    struct node *rchild;
};

  struct node *root = NULL;
  int a[max1];
  int n,i=0,count=0;

    struct node *createbt(int i);
    void inorder(struct node
    *root);
    void preorder(struct node *root);
    void postorder(struct node *root);

void main(void)
{
clrscr();
printf("\nenter the range of an array: ");
scanf("%d",&n);
printf("\nenter the list of array elements:
```

```
"); for(i=0;i<=n-1;i++)
scanf("%d",&a[i]);
```

```
printf("\nthe list of array elements before creating binary tree:
"); for(i=0;i<=n-1;i++)
printf("\na[%d]=%d\t%u",i,a[i],&a[i]);
printf("\n\n BINARY TREE USING ARRAYS IN INTEGERS FORMAT \n");
        i = 0;
        root = createbt(i);
printf("\n\t the total number of nodes in binary tree are %d\n\n",count);
        printf("\n In-Order Tree Traversal Using Recursion \n\t");
        inorder(root);
        printf("\n Pre-OrderTree Traversal Using Recursion \n\t");
        preorder(root);
        printf("\n Post-Order Tree Traversal Using Recursion \n\t");
        postorder(root);
printf("\nthe list of array elements after creating binary tree: ");
for(i=0;i<=n-1;i++)
printf("\na[%d]=%d\t%u",i,a[i],&a[i]);
getch();
}//main

 /* construct binary tree from Arrays */
 struct node *createbt(int i)
 {
        struct node *temp;
        if(i>=n)
        return NULL;
        temp = (struct node *)malloc(sizeof(struct node));
        count++;
        /* node at position i - have its left child at the position (2 * i) + 1 */
        temp->lchild = createbt((2 * i) + 1);
        temp->data  = a[i];
        /* node at position i - have right child at the position (2 * i) + 2 */
        temp->rchild = createbt((2 * i) + 2);
        return temp;
 }//createbt


 /* In-Order traversal in Binary Tree
 */ void inorder(struct node *root)
 {
        if (root!=NULL)
        {
                                        i
                                        n
                                        o
        }                               r
 }//inorder                             d
                                        e
```

r
(
r
o
o
t
-
>
l
c
h
i
l
d
)
;
p
r
i
n
t
f
(
"
%
d
"
,
r
o
o
t
-
>
d
a
t
a
)
;
i
n
o
r
d
e
r
(
r
o
o
t
-
>
r
c
h
i
l
d
)
;

```
/* Pre-Order traversal in Binary Tree */
void preorder(struct node *root)
{
        if (root!=NULL)
        {
                                        p
                                        r
                                        i
        }                               n
}//preorder                             t
                                        f
                                        (
                                        "
                                        %
                                        d
                                        "
                                        ,
                                        r
                                        o
                                        o
                                        t
                                        -
                                        >
                                        d
                                        a
                                        t
                                        a
                                        )
                                        ;
                                        p
                                        r
                                        e
                                        o
                                        r
                                        d
                                        e
                                        r
                                        (
                                        r
                                        o
                                        o
                                        t
                                        -
                                        >
                                        l
```

```
c                                   (
h                                   r
il                                  o
d                                   o
)                                   t
;                                   -
p                                   >
r                                   r
e                                   c
o                                   h
r                                   il
d                                   d
e                                   )
r                                   ;
```

```
/* Post-Order traversal in Binary Tree */
void postorder(struct node *root)
{
        if (root!=NULL)
        {
                postorder(root->lchild);
                postorder(root->rchild);
                printf(" %d ", root->data);
        }
}//postorder
```

6. **Write a C program that uses functions to perform the following operations on binary tree using arrays:**
    a. **Create a binary search tree of characters**
    b. **Traverse above binary search tree recursively**
        i. **in-order**
        ii. **pre-order**
        iii. **post-order**

```
//wap for binary tree - characters using arrays
#include<stdio.h>
#include<conio.h>
#define max1 100

struct node
{
    struct node *lchild;
    char data[max1][max1];
    struct node *rchild;
};
```

```
struct node *root = NULL;
```

```
  char a[max1][max1];
  int n,i=0,count=0;

    struct node *createbt(int i);
    void inorder(struct node
    *root);
    void preorder(struct node *root);
    void postorder(struct node *root);

void main(void)
{
clrscr();
printf("\nenter the range of an array: ");
scanf("%d",&n);
printf("\nenter the list of array elements:
"); for(i=0;i<=n-1;i++)
{
scanf("%s",a[i]);
}
printf("\nthe list of array elements before creating binary tree:
"); for(i=0;i<=n-1;i++)
{
printf("\na[%d]=%s\t%u",i,a[i],&a[i]);
}
printf("\n\n BINARY TREE USING ARRAYS IN CHARACTERS FORMAT
\n");
        i = 0;
        root = createbt(i);
printf("\n\t the total number of nodes in binary tree are %d\n\n",count);
        printf("\n In-Order Tree Traversal Using Recursion \n\t");
        inorder(root);
        printf("\n Pre-OrderTree Traversal Using Recursion \n\t");
        preorder(root);
        printf("\n Post-Order Tree Traversal Using Recursion \n\t");
        postorder(root);
printf("\n\nthe list of array elements after creating binary tree: \n");
for(i=0;i<=n-1;i++)
printf("\na[%d]=%s\t%u",i,a[i],&a[i]);
getch();
}//main

  /* construct binary tree from Arrays */
  struct node *createbt(int i)
  {
        struct node *temp;
        if(i>=n)
        return NULL;
```

```
        temp = (struct node *)malloc(sizeof(struct node));
        count++;
        /* node at position i - have its left child at the position (2 * i) + 1 */
        temp->lchild = createbt((2 * i) + 1);
        //temp->data = a[i];
    strcpy(temp->data,a[i])
    ;
        /* node at position i - have right child at the position (2 * i) + 2 */
        temp->rchild = createbt((2 * i) + 2);
        return temp;
}//createbt


/* In-Order traversal in Binary Tree
*/ void inorder(struct node *root)
{
        if (root!=NULL)
        {
                                        i
                                        n
                                        o
        }                               r
}//inorder                              d
                                        e
                                        r
                                        (
                                        r
                                        o
                                        o
                                        t
                                        -
                                        >
                                        l
                                        c
                                        h
                                        i
                                        l
                                        d
                                        )
                                        ;
                                        p
                                        r
                                        i
                                        n
                                        t
                                        f
                                        (
```

```
"%s",root->data);
inorder(root->rchild);
```

```
/* Pre-Order traversal in Binary Tree */
void preorder(struct node *root)
{
        if (root!=NULL)
        {
                printf("%s",root->da
        }
}//preorder
```

```
t
a
)
;
p
r
e
o
r
d
e
r
(
r
o
o
t
-
>
l
c
h
i
l
d
)
;
p
r
e
o
r
d
e
r
(
r
o
o
t
-
>
r
c
h
i
l
d
)
;
```

```
/* Post-Order traversal in Binary Tree */
void postorder(struct node *root)
{
        if (root!=NULL)
        {
                p
                o
                st
                o
                r
                d
                er
                (r
                o
                ot
                -
                >
                lc
                hi
                ld
                );
        }
}//postorder
```

p
o
st
o
r
d
er
(r
o
ot
-
>
rc
hi
ld
);
p
ri
nt
f(
"
%
s
",
r
o
ot
-
>
d
at
a)
;

7. **Write a C program that uses functions to perform the following operations on binary tree using double linked list:**
    a. **Create a binary tree of integers**
    b. **Traverse above binary tree non-recursively**
        i. **in-order**
        ii. **pre-order**
        iii. **post-order**

```
//wap for binary tree - nonrecursion using doubly linked list and templates
#include<stdio.h>
#include<conio.h
>
#include<stdlib.h
> struct node
{
int data;
struct node *left;
struct node *right;
};
struct node
*root=NULL,*temp=NULL,*temp1=NULL,*temp2=NULL,*parent=NULL;

        void create(void);
        void traverse(void);
        void insert(struct node *root,struct node *temp1);
        void inorder(struct node *root);
        void preorder(struct node *root);
        void postorder(struct node *root);
        void push(struct node *item,int *top,struct node *s[10]);
        int stempty(int top);
        void pop(int *top,struct node *s[10],struct node **curr);


void main(void)
{
int choice;
while(1)
{
clrscr();
printf("\n\n PROGRAM TO PERFORM VARIOUS OPERATIONS ON
BINARY TREE USING NON-RECURSION \n");
printf("\n\n WE ARE IN MAIN FUNCTION \n");
printf("1: CREATION/INSERTION\n");
printf("2: TRAVERSE\n");
printf("0: QUIT\n");
```

```
printf("\nenter ur choice (0-2) in main: ");
scanf("%d",&choice);
switch(choice)
{
case 1:
                                        c
                                        r
case 2:                                 e
                                        a
                                        t
case 0:                                 e
                                        (
                                        )
                                        ;
                                        b
                                        r
                                        e
                                        a
                                        k
                                        ;

                                        t
                                        r
                                        a
                                        v
                                        e
                                        r
                                        s
                                        e
                                        (
                                        )
                                        ;
                                        b
                                        r
                                        e
                                        a
                                        k
                                        ;

                                        e
                                        x
                                        i
                                        t
                                        (
                                        0
                                        )
                                        ;
```

```
b                                                   a
r                                                   k
e                                                   ;
        default:
                printf("\n Invalid choice");
                break;
        }//switch

        }//while
        getch();
        }//main

        void create(void)
        {
        int val;
        char ans;
        do
        {
        printf("\nenter the data: ");
        scanf("%d",&val);
        temp1=(struct node *)malloc(sizeof(struct node));    //allocate memory for node
        if(temp1==NULL)
        printf("\nunable to allocate memory");
        temp1->data=val;
        temp1->left=NULL;
        temp1->right=NULL;
        if(root==NULL)
        root=temp1;
        else
        insert(root,temp1);
        printf("\n do u want to add more nodes (y/n): ");
        ans=getche();
        }while(ans=='y' || ans=='Y');
        printf("\n\n\t binary tree nodes has been created successfully \n\n");
        }//create
```

```
void traverse(void)
{
if(root==NULL)
printf("\n binary tree is empty\n");
else
{
printf("\n\tThe non-recursive inorder traversal of a binary tree is: ");
inorder(root);
printf("\n\tThe non-recursive preorder traversal of a binary tree is: ");
preorder(root);
printf("\n\tThe non-recursive postorder traversal of a binary tree is: ");
postorder(root);
}
}//traverse


void insert(struct node *root,struct node *temp1)
{
char ch;
printf("\n\twhere to insert the node either in left/right child of %d root node :
",root->data);
ch=getche();
if((ch=='l')||(ch=='L'))
{
if((root->left)==NULL)
root->left=temp1;
else
insert(root->left,temp1);
}
else
{
if(root->right==NULL)
root->right=temp1;
else
insert(root->right,temp1);
}
}//insert

void push(struct node *item,int *top,struct node *s[10])
{
*top=*top+1;
s[*top]=item;
}//push

void pop(int *top, struct node *s[10],struct node **curr)
{
```

```
*curr=s[(*top)--];
}//pop


int stempty(int top)
{
if(top==-1)
return 1;
else
return 0;
}//stempty

void inorder(struct node *root)
{
struct node *curr,*s[10];
int top=-1;
if(root==NULL)
return;
curr=root;
for(;;)
{
while(curr!=NULL)
{
push(curr,&top,s);
curr=curr->left;
}
if((!stempty(top)))
{
pop(&top,s,&curr);
printf(" %d ",curr->data);
curr=curr->right;
}
else
return;
}
}//inorder

void preorder(struct node *root)
{
struct node *curr, *s[10];
int top=-1;
if(root==NULL)
return;
curr=root;
for(;;)
{
```

```
while(curr!=NULL)
{
printf(" %d ",curr->data);
push(curr,&top,s);
curr=curr->left;
}
if((!stempty(top)))
{
pop(&top,s,&curr);
curr=curr->right;
}
else
return;
}
}//preorder


void postorder(struct node *root)
{
struct stack
{
struct node *ele;         //here placing the node containing value
int check;      //check 1 means visiting left subtree, check 0 means visiting
right subtree
}st[10];
struct node *curr;
int top=-1;
if(root==NULL)
return;
curr=root;
for(;;)
{
while(curr!=NULL)
{
top++;
st[top].ele=curr;
st[top].check=1;          //visiting the left subbranch
curr=curr->left;
}
while(st[top].check==0)
{
curr=st[top].ele;
top--;
printf(" %d ",curr->data);
if(stempty(top))
return;
```

```
}
curr=st[top].ele; //pushing the element onto the stack
curr=curr->right;
st[top].check=0;    //visiting the right subbranch
}
}//postorder
```

8. **Write a C program that uses functions to perform the following operations on binary tree using double linked list:**
    a. **Create a binary tree of integers**
    b. **Traverse above binary tree recursively**
        i. **in-order**
        ii. **pre-order**
        iii. **post-order**

```
//wap for binary tree - recursion using doubly linked list and templates
#include<stdio.h>
#include<conio.h
>
#include<stdlib.h
> struct node
{
int data;
struct node *left;
struct node *right;
};
struct node
*root=NULL,*temp=NULL,*temp1=NULL,*temp2=NULL,*parent=NULL;

        void create(void);
        void traverse(void);
        void insert(struct node *root,struct node *temp1);
        void inorder(struct node *root);
        void preorder(struct node *root);
        void postorder(struct node *root);

void main(void)
{
int choice;
while(1)
{
clrscr();
printf("\n\n PROGRAM TO PERFORM VARIOUS OPERATIONS ON
BINARY TREE USING RECURSION \n");
printf("\n\n WE ARE IN MAIN FUNCTION \n");
```

```
printf("1: CREATION/INSERTION\n");
printf("2: TRAVERSE\n");
printf("0: QUIT\n");
printf("\nenter ur choice (0-2) in main: ");
scanf("%d",&choice);
switch(choice)
{
case 1:
                                        c
                                        r
case 2:                                 e
                                        a
                                        t
case 0:                                 e
                                        (
                                        )
                                        ;
                                        b
                                        r
                                        e
                                        a
                                        k
                                        ;

                                        t
                                        r
                                        a
                                        v
                                        e
                                        r
                                        s
                                        e
                                        (
                                        )
                                        ;
                                        b
                                        r
                                        e
                                        a
                                        k
                                        ;

                                        e
                                        x
                                        i
                                        t
                                        (
```

```
0                                                   e
)                                                   a
;                                                   k
b                                                   ;
r
        default:
                printf("\n Invalid choice");
                break;
        }//switch

        }//while
        getch();
        }//main

        void create(void)
        {
        int val;
        char ans;
        do
        {
        printf("\nenter the data: ");
        scanf("%d",&val);
        temp1=(struct node *)malloc(sizeof(struct node));    //allocate memory for node
        if(temp1==NULL)
        printf("\nunable to allocate memory");
        temp1->data=val;
        temp1->left=NULL;
        temp1->right=NULL;
        if(root==NULL)
        root=temp1;
        else
        insert(root,temp1);
        printf("\n do u want to add more nodes (y/n): ");
        ans=getche();
        }while(ans=='y' || ans=='Y');
```

```
printf("\n\n\t binary tree nodes has been created successfully \n\n");
}//create

void traverse(void)
{
if(root==NULL)
printf("\n binary tree is empty\n");
else
{
printf("\n\tThe recursive inorder traversal of a binary tree is: ");
inorder(root);
printf("\n\tThe recursive preorder traversal of a binary tree is: ");
preorder(root);
printf("\n\tThe recursive postorder traversal of a binary tree is: ");
postorder(root);
}
}//traverse


void insert(struct node *root,struct node *temp1)
{
char ch;
printf("\n\twhere to insert the node either in left/right child of %d root node :
",root->data);
ch=getche();
if((ch=='l')||(ch=='L'))
{
if((root->left)==NULL)
root->left=temp1;
else
insert(root->left,temp1);
}
else
{
if(root->right==NULL)
root->right=temp1;
else
insert(root->right,temp1);
}
}//insert

void inorder(struct node *root)
{
if(root!=NULL)
{
inorder(root->left);
```

```
printf("%d ",root->data);
inorder(root->right);
}
}//inorder

void preorder(struct node *root)
{
if(root!=NULL)
{
printf("%d ",root->data);
preorder(root->left);
preorder(root->right);
}
}//preorder

void postorder(struct node *root)
{
if(root!=NULL)
{
postorder(root->left);
postorder(root->right);
printf("%d ",root->data);
}
}//postorder
```

9. **Write a C program that uses functions to perform the following operations on binary search tree using double linked list:**
    a. **Create a binary search tree of integers**
    b. **Traverse above binary search tree recursively**
        i. **in-order**
        ii. **pre-order**
        iii. **post-order**

```
//wap for binary search tree using doubly linked list and
templates #include<stdio.h>
#include<conio.h
>
#include<stdlib.h
> struct node
{
int data;
struct node *left;
struct node *right;
};
```

```
struct node
*root=NULL,*temp=NULL,*temp1=NULL,*temp2=NULL,*parent=NULL;

        void create(void);
        void traverse(void);
        void insert(struct node *root,struct node *temp1);
        void delet(void);
        void del(struct node *root,int key);
        void search(void);
        void find(struct node **temp1,int key,struct node **parent);
        void inorder(struct node *root);
        void preorder(struct node *root);
        void postorder(struct node *root);

void main(void)
{
int choice;

while(1)
{
clrscr();
printf("\n\n PROGRAM TO PERFORM VARIOUS OPERATIONS ON
BINARY SEARCH TREE (INTEGERS) \n");
printf("\n\n WE ARE IN MAIN FUNCTION \n");
printf("1: CREATION/INSERTION\n");
printf("2: TRAVERSE\n");
printf("3: DELETION\n");
printf("4: SEARCH\n");
printf("0: QUIT\n");
printf("\nenter ur choice (0-4) in main: ");
scanf("%d",&choice);
switch(choice)
{
case 1:
                                        c
                                        r
case 2:                                 e
                                        a
                                        t
case 3:                                 e
                                        (
                                        )
```

;
b
r
e
a
k
;

t
r
a
v
e
r
s
e
(
)
;
b
r
e
a
k
;

delet();

```
                                        break;
case 4:
                                        s
                                        e
                                        a
case 0:                                 r
                                        c
                                        h
                                        (
                                        )
                                        ;
                                        b
                                        r
                                        e
                                        a
                                        k
                                        ;

                                        e
                                        x
                                        i
                                        t
                                        (
                                        0
                                        )
                                        ;
                                        b
                                        r
                                        e
                                        a
                                        k
                                        ;
default:
        printf("\n Invalid choice");
        break;
}//switch
}//while
getch();
}//main

void create(void)
```

```
{
int val;
char ans;
do
{
printf("\nenter the data: ");
scanf("%d",&val);
temp1=(struct node *)malloc(sizeof(struct node));    //allocate memory for node
if(temp1==NULL)
printf("\nunable to allocate memory");
temp1->data=val;
temp1->left=NULL;
temp1->right=NULL;
if(root==NULL)
root=temp1;
else
insert(root,temp1);
printf("\n do u want to add more nodes (y/n): ");
ans=getche();
}while(ans=='y' || ans=='Y');
printf("\n\n\t binary search tree nodes has been created successfully \n\n");
}//create

void traverse(void)
```

```
{
if(root==NULL)
printf("\n binary search tree is empty\n");
else
{
printf("\n\tThe recursive inorder traversal of a binary search tree is: ");
inorder(root);
printf("\n\tThe recursive preorder traversal of a binary search tree is: ");
preorder(root);
printf("\n\tThe recursive postorder traversal of a binary search tree is: ");
postorder(root);
}
}//traverse


void insert(struct node *root,struct node *temp1)
{
if((temp1->data)<(root->data))
{
if((root->left)==NULL)
root->left=temp1;
else
insert(root->left,temp1);
}
if((temp1->data)>(root->data))
{
if(root->right==NULL)
root->right=temp1;
else
insert(root->right,temp1);
}
}//insert

void inorder(struct node *root)
{
if(root!=NULL)
{
inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
```

```
}
}//inorder

void preorder(struct node *root)
{
if(root!=NULL)
{
printf("%d ",root->data);
preorder(root->left);
preorder(root->right);
}
}//preorder

void postorder(struct node *root)
{
if(root!=NULL)
{
postorder(root->left);
postorder(root->right);
printf("%d ",root->data);
}
}//postorder

void search(void)
{
int key;
printf("\n Enter the element which u want to search: ");
scanf("%d",&key);
temp2=root;
find(&temp2,key,&parent);
if(temp2==NULL)
printf("\n searched element is not present");
else
printf("\n Parent of node %d is %d",temp2->data,parent->data);
}//search

void find(struct node **temp1,int key,struct node **parent)
{
if(*temp1==NULL)
printf("\n binary search tree is empty \n");
```

```
else
{
while(*temp1!=NULL)
{
if(((*temp1)->data)==key)
{
printf("\n The %d searched element is present\n",((*temp1)->data));
break;
}
*parent=*temp1;        //stores the parent value
if(((*temp1)->data) > key)
*temp1=(*temp1)->left;
else
*temp1=(*temp1)->right;
}
}
return;
}//find


void delet(void)
{
int key;
printf("\n Enter the element which u want to delete: ");
scanf("%d",&key);
if(key==root->data)
{
root=NULL;     //assigning a value NULL to root
}
else
del(root, key);
}//delet


void del(struct node *root, int key)
{
if(root==NULL)
printf("\n binary search tree is empty\n");
else
{
```

```
temp2=root;
find(&temp2,key,&parent);
//temp2 node is to be deleted
//deleting a node with two children
if(temp2->left!=NULL && temp2->right!=NULL)
{
parent=temp2;
temp=temp2->right;
while(temp->left!=NULL
)
{
parent=temp;
temp=temp->left;
}
temp2->data=temp->data;
//copying the immediate
successor temp2->right=NULL;
printf("\n Now Deleted it");
return;
}
//deleting a node having only one child
//the node to be deleted has left child
if(temp2->left!=NULL && temp2->right==NULL)
{
if(parent->left==temp2)
parent->left=temp2->left;
else
parent->right=temp2->left
; temp2=NULL;
free(temp2);
printf("\n Now Deleted it");
return;
}
//the node to be deleted has right child
if(temp2->left==NULL && temp2->right!=NULL)
{
if(parent->left==temp2)
parent->left=temp2->right;
else
parent->right=temp2->right;
temp2=NULL;
```

```
free(temp2);
printf("\n Now Deleted it");
return;
}
//deleting a node which is having no child
if(temp2->left==NULL && temp2->right==NULL)
{
if(parent->left==temp2)
parent->left=NULL;
else
parent->right=NULL;
printf("\n Now Deleted it");
return;
}
}
}//del
```